POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

INFORMATION MANAGEMENT
Individual Project

Automobile Repair Shop Management System

PHP-Laravel with MySQL Database

Prepared for: Information Management

By: **Kim Phillip G. Andador**

DIT 2-1

Date: June 2026

Table of Contents

Case Study Context

The project is based on the following case study:

> *"You are designing a database for an automobile repair shop. When a customer brings in a vehicle, a service advisor will write up a repair order. This order will identify the customer and the vehicle, along with the date of service and the name of the advisor. A vehicle might need several different types of service in a single visit. These could include oil change, lubrication, rotate tires, and so on. Each type of service is billed at a pre-determined number of hours work, regardless of the actual time spent by the technician. Each type of service also has a flat book rate of dollars-per-hour that is charged."*

This case study defines the core business rules for the system:

Customer-to-Vehicle Relationship: A customer may own multiple vehicles, and each vehicle belongs to exactly one customer.

Repair Order as Central Document: Each service visit generates a repair order that ties together the customer, vehicle, advisor, and date.

Multiple Services Per Visit: A single repair order can include several different service types (oil change, tire rotation, transmission service, etc.).

Pre-Determined Book Hours: Each service type is billed at a fixed number of hours, not the actual time spent by the technician.

Flat Rate Per Hour: Each service type has a standard dollars-per-hour rate. The line total is calculated as book hours multiplied by rate per hour.

Project Requirements

The following requirements were specified for the Information Management individual project:

MySQL Database: The system must use a MySQL database with proper schema design, including tables, foreign keys, and indexes.

PHP-Laravel Integration: The application must be built using the PHP-Laravel framework with MVC architecture.

Login Authentication: A login form with authentication that validates user credentials against the database.

Landing Page & CRUD Modules: A dashboard landing page with maintenance modules providing Create, Read, Update, and Delete operations.

Additional Features: Extra features may be added as necessary to enhance the system.

Audit Trail: A comprehensive audit trail that logs all significant actions (create, update, delete, login, logout).

Role-Based Permissions: Role-based access control with different permission levels for different user roles.

Database Schema

The database is named im_indivproject and runs on MySQL 8. It contains nine tables that collectively implement the case study business rules and project requirements.

3.1 Table Overview

Table	Description
users	Stores login credentials (first_name, last_name, email, password, is_active, user_type).
roles	Defines the four hierarchical roles: admin (level 4), manager (3), staff (2), customer (1).
role_user	Pivot table linking users to roles (many-to-many).

customers	Stores client information (first_name, last_name, email, phone, address).
vehicles	Stores vehicle details (make, model, year, license_plate, VIN). FK to customers.
service_types	Service catalog with name, book_hours, rate_per_hour.
repair_orders	Core business document. Links to customer and vehicle.
repair_order_services	Junction connecting repair orders to service types with line_total.
audit_logs	Records all actions: user_id, action, entity_type, entity_id, summary, old/new values.

3.2 Database Migrations

The schema is version-controlled through 16 Laravel migration files. All core business tables use InnoDB with proper foreign key constraints and strategic indexes.

3.3 Conceptual Data Model

The conceptual data model identifies the four core business entities — Customer, Vehicle, Service, and Repair Order — and their relationships independent of any technology platform. This model was derived directly from analysis of the case study narrative.

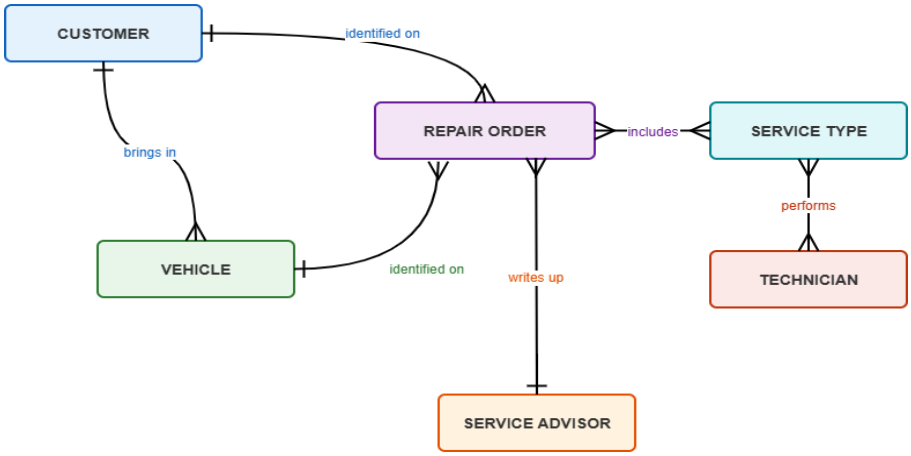

Figure 1: Conceptual Data Model — high-level business entities and relationships

3.4 Logical Data Model

The logical data model expands each entity into a detailed relational structure with specific attributes, primary keys, and foreign keys. The repair_order_service junction table resolves the many-to-many relationship between Repair Order and Service. All attributes have defined domain-appropriate data types.

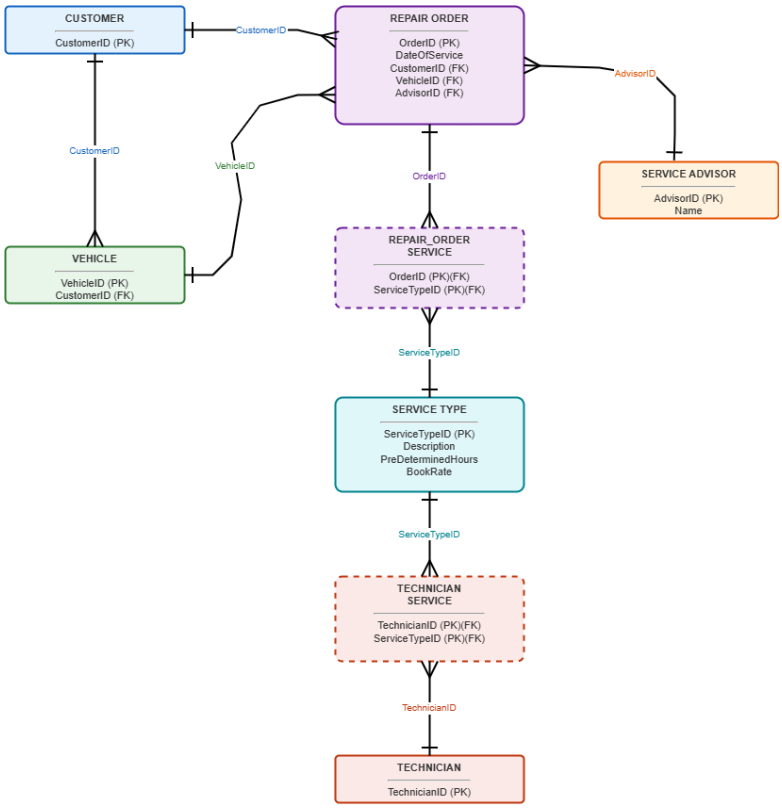

Figure 2: Logical Data Model — entities with attributes, primary keys, and foreign keys

3.5 Physical Data Model

The physical data model shows the MySQL 8 implementation with exact column names, data types, nullable constraints, default values, foreign key definitions, and indexes. This model is the direct blueprint for the 16 Laravel migration files that create the im_indivproject database.

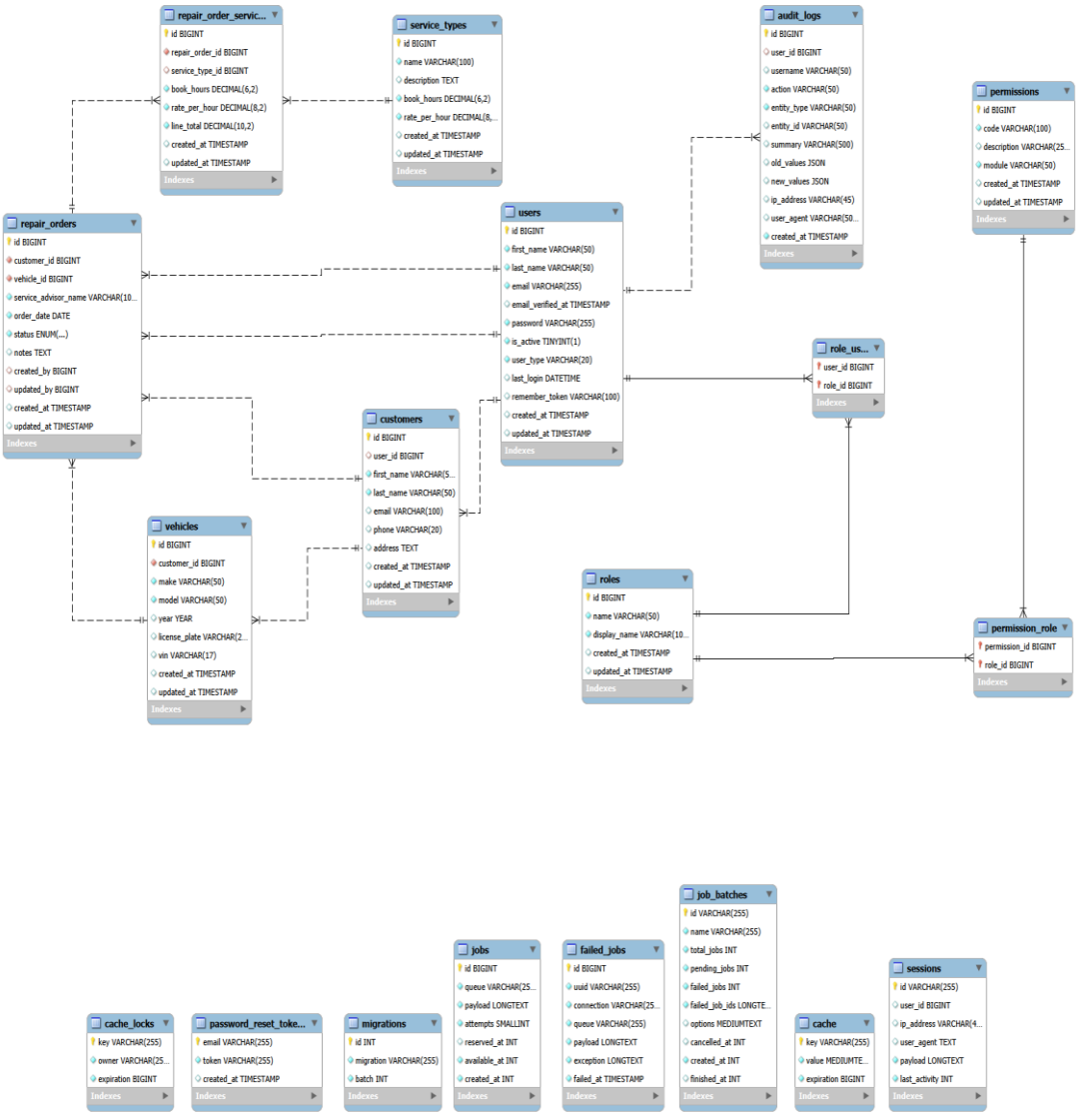

Figure 3: Physical Data Model — MySQL 8 implementation with exact column types and constraints

Business Objects (Entities)

The system models seven core business entities that directly map to real-world concepts in the automobile repair shop.

4.1 User

Business Purpose: Represents every person who can access the system.

- User ID
- First Name, Last Name
- Email (login credential)
- Password (hashed)
- Is Active (status flag)
- User Type (customer/staff)
- Timestamps

4.2 Role

Business Purpose: Defines the permission level. Each role has a numeric hierarchy level (4=admin, 3=manager, 2=staff, 1=customer).

- Role ID
- Name (admin/manager/staff/customer)
- Display Name
- Timestamps

4.3 Customer

Business Purpose: Represents a client of the auto repair shop. May own multiple vehicles.

- Customer ID
- First Name, Last Name
- Email, Phone, Address
- Linked User ID (optional)
- Timestamps

4.4 Vehicle

Business Purpose: Represents a car brought in for service. Owned by exactly one customer.

- Vehicle ID
- Customer ID (owner)
- Make, Model, Year
- License Plate, VIN
- Timestamps

4.5 Service Type

Business Purpose: Represents a service offered. Implements pre-determined hours and flat rate per hour.

- Service Type ID
- Name, Description
- Book Hours (fixed)
- Rate Per Hour (flat rate)
- Timestamps

4.6 Repair Order

Business Purpose: The central document representing a single service visit.

- Repair Order ID
- Customer ID, Vehicle ID
- Service Advisor Name, Order Date
- Status (open/in_progress/completed/cancelled)
- Notes, Created By, Updated By
- Timestamps

4.7 Service Line

Business Purpose: A single service item on a repair order with snapshotted pricing.

- Service Line ID
- Repair Order ID, Service Type ID
- Book Hours, Rate Per Hour
- Line Total (calculated)
- Timestamps

4.8 Audit Log

Business Purpose: Records every significant action for accountability. Read-only, admin-only.

- Audit Log ID
- User ID, Username
- Action, Entity Type, Entity ID
- Summary, Old/New Values (JSON)
- IP Address, User Agent, Timestamp

Relationships Between Entity Objects

The business entities are interconnected through well-defined relationships that mirror real-world repair shop operations.

5.1 Customer ↔ User (Optional One-to-One)

Type & Mechanism: Optional one-to-one. A customer may optionally be linked to a user account. When a new user registers, both a user account (customer role) and a customer record are created automatically.

Example: Supports the self-service customer portal. Kim Phillip Gaton has both a user account and a customer record.

5.2 User ↔ Role (Many-to-Many)

Type & Mechanism: Many-to-many via role_user pivot table. Each user has one role. The role determines navigation, controller access, and data visibility.

Example: admin (level 4) sees all links. Staff (level 2) cannot access Users or Audit.

5.3 Customer ↔ Vehicle (One-to-Many)

Type & Mechanism: One-to-many. A customer can own multiple vehicles. Each vehicle has a customer_id FK. Deleting a customer cascades to delete all their vehicles.

Example: Pedro Gonzales owns two vehicles. Both appear under his customer record.

5.4 Customer ↔ Repair Order (One-to-Many)

Type & Mechanism: One-to-many. A customer can have multiple repair orders over time, creating a complete service history.

Example: Enables service history view. Customers see their own order history.

5.5 Vehicle ↔ Repair Order (One-to-Many)

Type & Mechanism: One-to-many. A vehicle can appear on multiple repair orders over its lifetime.

Example: Allows the shop to track maintenance patterns for a specific vehicle.

5.6 Repair Order ↔ Service Line (One-to-Many)

Type & Mechanism: One-to-many (CRITICAL). A repair order can have multiple service line items. Directly implements the case study: "a vehicle might need several different types of service." Each line itemized with book hours, rate, and total.

Example: Repair Order #1 includes 4 services: Air Filter ($24.00), Rotate Tires ($45.00), Transmission Service ($165.00) = Grand Total $234.00.

5.7 Service Type ↔ Service Line (One-to-Many)

Type & Mechanism: One-to-many. A service type can appear on many different repair orders. Book hours and rate snaphotted at creation time.

Example: Preserves historical pricing even if rates change.

5.8 User ↔ Audit Log (One-to-Many)

Type & Mechanism: One-to-many. Every action generates an audit log entry including login/logout.

Example: Shows who, when, and what the old/new values were.

5.9 Summary Relationship Diagram

The following summarizes all entity relationships:

- Customer (1) ---< (N) Vehicle: "A customer owns many vehicles"
- Customer (1) ---< (N) Repair Order: "A customer has many service visits"
- Vehicle (1) ---< (N) Repair Order: "A vehicle has many service visits"
- Repair Order (1) ---< (N) Service Line: "A repair order has many services"
- Service Type (1) ---< (N) Service Line: "A service type appears on many orders"
- Customer (0..1) --- (0..1) User: "A customer may optionally have a login"
- User (N) ---< (N) Role: "Users are assigned roles (via role_user pivot)"
- User (1) ---< (N) Audit Log: "A user creates many audit trail entries"

Application Architecture

The application follows MVC as implemented by Laravel. Key components:

Controllers (7): Auth, Customer, Vehicle, RepairOrder, ServiceType, Users, Audit, Dashboard

Blade Views (23): Login, Dashboard (2), Customers (3), Vehicles (3), Repair Orders (4), Service Types (3), Users (3), Audit (1), Template (3)

Form Requests (5): Register, Customer, Vehicle, ServiceType, StoreUser, UpdateUser

Middleware (1): SessionAuth - checks for user_id in session

Helpers (1): AuditHelper - static log() and logAuth() for audit trail

Routes (20+): Guest routes (login, register), authenticated routes (logout, dashboard, 5 resource controllers, audit, AJAX vehicle loader)

Frontend: Tailwind CSS via Vite (46.93 kB). Responsive design with mobile nav menu.

Authentication & Role Management

7.1 Login Flow

Email and password validated against users table using Hash::check(). On success, user info and role stored in session. On failure, error message displayed. Login/logout recorded in audit trail.

7.2 Registration

New users register via registration form. Auto-assigned "customer" role with corresponding customer record created. Enables self-service portal.

7.3 Session Storage

Session stores: user_id, first_name, last_name, email, role (name string), role_name (display name). Session drives all authorization via session("role") lookups. Role populated from role_user/roles join on login.

CRUD Maintenance Modules

Five full CRUD modules provide create, read (list), update, and delete functionality:

Module	Methods	Description
Customers	index, create, store, edit, update, destroy	Manages client records.
Vehicles	index (customer_id filter), create, store, edit, update, destroy	Vehicle records linked to customers.
Repair Orders	index, create, store, show, edit, update, destroy,	Core module with AJAX customer->vehicle loading and calculated totals.

	removeService, getByCustomer	
Service Types	index, create, store, edit, update, destroy	Service catalog with book_hours and rate_per_hour.
Users	index, create, store, edit, update, destroy	User admin with role hierarchy enforcement.

8.1 Repair Order Creation Flow

The Repair Order create page demonstrates the full case study implementation:

1. Select customer from dropdown - triggers AJAX to load that customer's vehicles dynamically.
2. Select vehicle from dynamically populated dropdown.
3. Enter service advisor name (defaults to logged-in user).
4. Set order date.
5. Check one or more service types - each shows name, book hours, and rate per hour.
6. Submit - system creates order and each service as a repair_order_service row with calculated line_total.
7. Show page displays all line items and grand total.

Audit Trail System

Accessible only to admin users. Logs all significant actions.

9.1 Audit Filters

- Entity Type filter (Auth, Customer, Vehicle, etc.)
- Action filter (Create, Update, Delete, Login, Logout)
- User filter
- Results paginated (50/page), ordered by most recent
- Rows color-coded by action type

9.2 Entities Tracked

Auth, Customers, Vehicles, Repair orders, Repair order services, Service types, Role user, and Users.

Role-Based Permissions

Permissions enforced at three levels throughout the application.

10.1 Role Hierarchy

Role	Level	Description
Admin	4	Full access - all modules, audit, users
Manager	3	All except audit; manages users below level 3
Staff	2	Customers, vehicles, orders, services only
Customer	1	Self-service - own vehicles and orders only

10.2 Enforcement Layers

Navigation Bar: Menu items conditionally rendered per role.

Controller Authorization: Every controller checks session("role"). Role gates on every CRUD method.

Form Request Validation: StoreUserRequest and UpdateUserRequest enforce hierarchy in role assignment.

Role Hierarchy Guards: UsersController guards edit/update/destroy by comparing target user role.

Case Study-to-System Mapping

The following maps each case study requirement to its implementation:

Case Study Requirement	Database	Application
Customer brings in a vehicle	customers + vehicles linked by FK	Customer to Vehicle in create/edit forms
Advisor writes repair order	repair_orders.service_advisor_name	Repair Order create form has advisor field
Order identifies customer, vehicle, date, advisor	customer_id, vehicle_id, order_date, service_advisor_name	Repair Order show page displays all four
Multiple services per visit	repair_order_services junction table	Checkbox service selection in order form
Pre-determined hours	service_types.book_hours	Shown in catalog and order line items
Flat book rate ($/hr)	service_types.rate_per_hour	Shown in catalog and order line items
Line total = hours x rate	repair_order_services.line_total (calculated)	Grand total on repair order show page
System tracks who did what	audit_logs table	Audit trail page with filters

Testing & Verification

12.1 Automated Tests

PHPUnit tests for SessionAuth middleware pass successfully.

12.2 Build Verification

npm run build produces 46.93 kB CSS from Tailwind via Vite.

12.3 Playwright Browser Verification

All case study requirements verified via automated browser testing:

- Login auth - credentials validated, session populated
- Dashboard - all role-appropriate links visible
- Repair Order #1: customer, vehicle, advisor, 4 services, calculated total
- Audit trail - 42 entries, 8 entity types, 3 filters
- Customers CRUD - 5 records with edit/delete
- All 5 CRUD modules verified

Conclusion

The Automobile Repair Shop Management System successfully implements all requirements:

MySQL Database: Normalized 9-table database with FKs and indexes.

PHP-Laravel Integration: MVC with 7 controllers, 23 views, form requests, middleware.

Login Authentication: Session-based auth with registration and role assignment.

CRUD Modules: Five full modules with create, read, update, delete.

Audit Trail: Comprehensive logging with entity/action/user filters.

Role Permissions: Four-tier hierarchy enforced at nav, controller, and data levels.

All case study business rules are faithfully implemented and verified through automated testing and browser-based verification.